

RESPONSE TIME AND THROUGHPUT

- Response time (Latency)
 - How long it takes to do a task.
 - Execution time; Cycles per Instruction (CPI); Avg. Memory access time (AMAT)
- Throughput
 - Total work done per unit time.
 - e.g., tasks/transactions/... per hour
 - Transactions per second (TPS); Million Instructions per second (MIPS)
- How are response time and throughput are affected by
 - Replacing the processor with a faster version?
 - Adding more processors?

13-Jan-2026

Recap:

- *What do we mean by Performance?*
- *What are the metrics typically used to express Performance?*

Agenda for Today:

- How do we compare the Performance of two Systems?
- More precise definition of Performance.
- Characterize Performance in view of COA.
- Benchmarking

PERFORMANCE AND RELATIVE PERFORMANCE

- Define Performance = *Time/cost/power needed to do a unit work* **<Lower the better>**

OR = Unit of Things done per unit time. **<Higher the better>**

$$\text{Performance}(x) = \text{Execution time}(x) \text{ seconds}$$

$$\text{Performance}(x) = 1 / \text{Execution time}(x) \text{ seconds}^{(-1)}$$

- Relative Performance: “X is *n* time faster than Y” or “X is *n* times as fast as Y” or “From Y to X the speedup is *n* times ”

$$\begin{aligned} \text{Speedup}(x/y) &= \text{Performance}(x) / \text{Performance}(y) \\ &= \text{Execution time}(y) / \text{Execution time}(x) \end{aligned}$$

- Example: time taken to run a program (*same program on different machines*)
 - Program A takes **10s** on Machine **X**, and **15s** on Machine **Y**
 - Execution Time(**Y**) / Execution Time(**X**) = 15s/10s = 1.5
 - So **X** is 1.5 times faster than **Y**; **Speedup(X/Y) = 1.5**

How do we measure the
Execution time of a Program?

1. Elapsed time

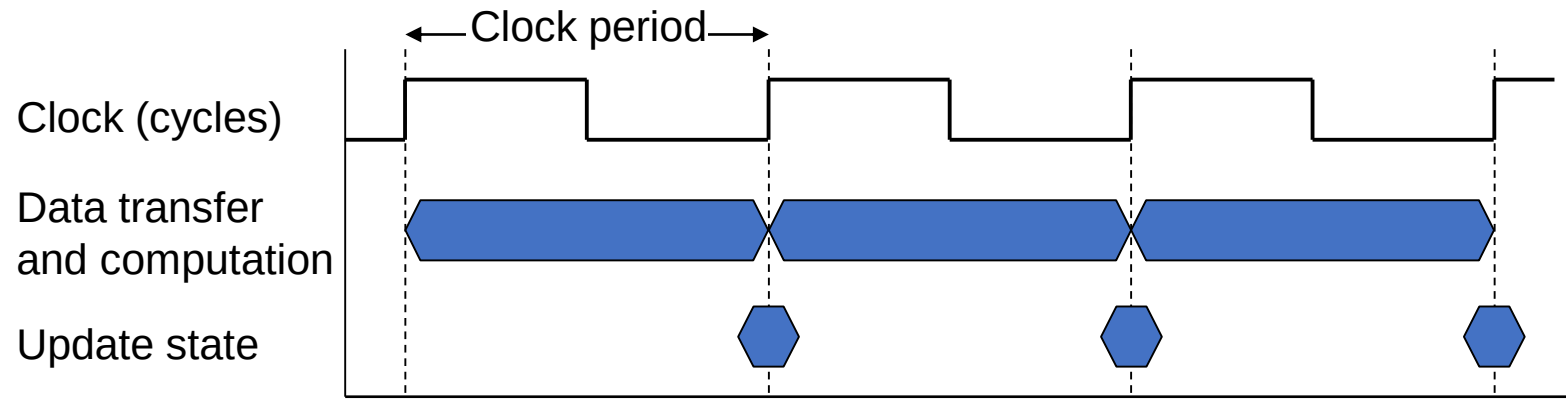
- Total response time, including all aspects in the system.
 - Processing, I/O, OS overhead, idle time
- Determines the overall system performance

2. CPU time

- Time spent processing a given job
 - Discounts I/O time; other jobs' share of elapsed time
- CPU time comprises of two components (i) user CPU time and (ii) system CPU time
 - **User CPU time**: Time spent in the user program (all the processing in the user space).
 - **System CPU time**: Time spent in the system (OS) for doing tasks on behalf of user program.
- Different programs are affected differently by CPU and system performance

CPU CLOCKING

- Operation of digital hardware is governed by a **constant-rate** Clock



- Clock period: duration of a clock cycle
 - e.g., $250\text{ps} = 0.25\text{ns} = 250 \times 10^{-12}\text{s}$
- Clock frequency (rate): cycles per second
 - e.g., $4.0\text{GHz} = 4000\text{MHz} = 4.0 \times 10^9\text{Hz}$

<https://www.intel.in/content/www/in/en/gaming/resources/cpu-clock-speed.html>

CPU TIME – MORE FINE-GRAINED VIEW

$$\begin{aligned}\text{CPU Time} &= \text{CPU Clock Cycles} \times \text{Clock Cycle Time} \\ &= \frac{\text{CPU Clock Cycles} \downarrow}{\text{Clock Rate} \uparrow}\end{aligned}$$

- Performance can be improved (Lower the CPU Time) by:
 - Reducing number of CPU clock cycles (**good algorithm** or **hardware design**)
 - Increasing clock rate (**good technology**) or Reducing the Clock Cycle Time

HARDWARE DESIGNER'S PERSPECTIVE

Let's say "Computer A": with 2GHz clock, takes 10s CPU time

- Goal: Designing a Faster "Computer B" that would take only 6s CPU time.
 - Target for 6s CPU time
 - Consider that Computer B causes 20% increase in clock cycles. (i.e. $1.2 \times \text{clock cycles}(A)$)
- How fast must Computer B clock be?

$$\text{Clock Rate}_B = \frac{\text{Clock Cycles}_B}{\text{CPU Time}_B} = \frac{1.2 \times \text{Clock Cycles}_A}{6s}$$

$$\begin{aligned}\text{Clock Cycles}_A &= \text{CPU Time}_A \times \text{Clock Rate}_A \\ &= 10s \times 2\text{GHz} = 20 \times 10^9\end{aligned}$$

$$\text{Clock Rate}_B = \frac{1.2 \times 20 \times 10^9}{6s} = \frac{24 \times 10^9}{6s} = 4\text{GHz}$$

INSTRUCTION COUNT, CPI AND MIPS

$$\text{CPU Time} = \text{CPU Clock Cycles} \times \text{Clock Cycle Time}$$

$$\text{Clock Cycles} = \text{Instruction Count (IC)} \times \text{Cycles per Instruction (CPI)}$$

$$\text{CPU Time} = \text{IC} \times \text{CPI} \times \text{Clock Cycle Time}$$

$$= \frac{\text{IC} \times \text{CPI}}{\text{Clock Rate}}$$

- Instruction Count (IC) for a program
 - *Is determined by program, ISA and compiler*
- Average cycles per instruction (CPI)
 - *Is often determined by CPU hardware and hw-sw interface*

$$\text{MIPS} = \text{Instruction Count} / (\text{Execution Time} \times 10^6)$$

- If *different instructions have different CPI*
 - *Average CPI is affected by instruction mix*

INSTRUCTION COUNT AND CPI - EXAMPLE

- Computer A: Cycle Time = 250ps, CPI = 2.0
- Computer B: Cycle Time = 500ps, CPI = 1.2
- Consider Same ISA -- Which is faster, and by how much?

$$\text{CPU Time}_A = \text{Instruction Count} \times \text{CPI}_A \times \text{Cycle Time}_A$$

$$= IC \times 2.0 \times 250\text{ps} = 1 \times 500\text{ps}$$

A is faster...

$$\text{CPU Time}_B = IC \times \text{CPI}_B \times \text{Cycle Time}_B$$

$$= 1 \times 1.2 \times 500\text{ps} = 1 \times 600\text{ps}$$

$$\frac{\text{CPU Time}_B}{\text{CPU Time}_A} = \frac{1 \times 600\text{ps}}{1 \times 500\text{ps}} = 1.2$$

...by this much

- If different instruction classes take different numbers of cycles

$$\text{Clock Cycles} = \sum_{i=1}^n (\text{CPI}_i \times \text{Instruction Count}_i)$$

Weighted average CPI

$$\text{CPI} = \frac{\text{Clock Cycles}}{\text{Instruction Count}} = \sum_{i=1}^n \left(\text{CPI}_i \times \underbrace{\frac{\text{Instruction Count}_i}{\text{Instruction Count}}}_{\text{Relative frequency}} \right)$$

INFLUENCE ON CPI – COMPILER AND PROGRAMMERS CHOICE

- Consider compiled code sequences using instructions in classes A, B, C
- On two machines with Clock Rate of 1Ghz

Class	A	B	C
CPI for class	1	2	3
IC in sequence 1	2	1	2
IC in sequence 2	4	1	1

■ Sequence 1: IC = 5

- Clock Cycles
 $= 2 \times 1 + 1 \times 2 + 2 \times 3$
 $= 10$
- Avg. CPI = $10/5 = 2.0$

$$\text{Ex Time} = (5) * (2) * (10^{-9})\text{s} = 10\text{ns}$$

$$\text{MIPS} = (1 \times 10^9) / (2) * (10^{-6}) = 500 \text{ MIPS}$$

■ Sequence 2: IC = 6

- Clock Cycles
 $= 4 \times 1 + 1 \times 2 + 1 \times 3$
 $= 9$
- Avg. CPI = $9/6 = 1.5$

$$\text{Ex Time} = (6) * (1.5) * (10^{-9})\text{s} = 9\text{ns}$$

$$\text{MIPS} = (1 \times 10^9) / (1.5) * (10^{-6}) = 667 \text{ MIPS}$$

The BIG Picture

$$\text{CPU Time} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Clock cycles}}{\text{Instruction}} \times \frac{\text{Seconds}}{\text{Clock cycle}}$$

$$\text{CPU Time} = IC \times CPI \times T_c$$

- Performance depends on different aspects:
 - **Algorithm**: affects Instruction count (IC), possibly CPI
 - **Programming language**: affects IC, CPI
 - **Compiler**: affects IC, CPI
 - **Instruction set architecture**: affects IC, CPI, T_c
 - **Computer Organization**: affects CPI, T_c

How can we reliably quantify the measure of Performance? *And What to measure?*

*Course Content: Chapter 1.9 and 1.10 from COAHP
Part from Chapter 2 (COAWS)*

BENCHMARK PRINCIPLES AND DESIRABLE CHARACTERISTICS

- Desirable characteristics of a benchmark program:
 - I. It is written in a high-level language, making it portable across different machines.
 - II. It is representative of a particular kind of programming domain or paradigm, such as systems programming, numerical programming, or commercial programming.
 - III. It can be measured easily. *<should not require complex tool-chains>*
 - IV. It has wide distribution and be reliably accessible.

Paper : [An Overview of Common Benchmarks](#)

BENCHMARK SUITE AND STANDARDS

- Benchmark suite
 - A collection of programs, defined in a high-level language.
 - Together attempt to provide a representative test of a computer in a particular application or system programming area
- SPEC – System Performance Evaluation Corporation
 - An industry consortium – computer hardware vendors (majority)
 - Defines and maintains the best-known collection of benchmark suites aimed at evaluating computer systems
 - Widely used for comparison and research purposes; *must pay to get them.*
- TPC – Transactions Processing Council
 - An industry consortium (Hardware and Software vendors)
 - Defines and maintains the best-known collection of benchmark suites aimed at evaluating computer systems
 - Widely used for comparison and research purposes; ***Available free of cost.***

Recommended Reading: <http://ldbcouncil.org/public/why-benchmarking>

SPEC CPU2017

- Latest known SPEC benchmark suite for CPU intensive workloads
- Industry standard suite for processor intensive applications.
- Appropriate for measuring performance for applications that spend most of their time doing computation rather than I/O.
- It is the Sixth generation of processor intensive suites from SPEC



- Consists of 43 benchmark programs in 4 different sub suites:
 - SpeedInteger & SpeedFP: measure the time to complete single tasks.
 - Rate Integer & RateFP: measure the throughput or work per unit time.

SPEC CPU 2006 INTEGER AND FLOATING-POINT BENCHMARKS

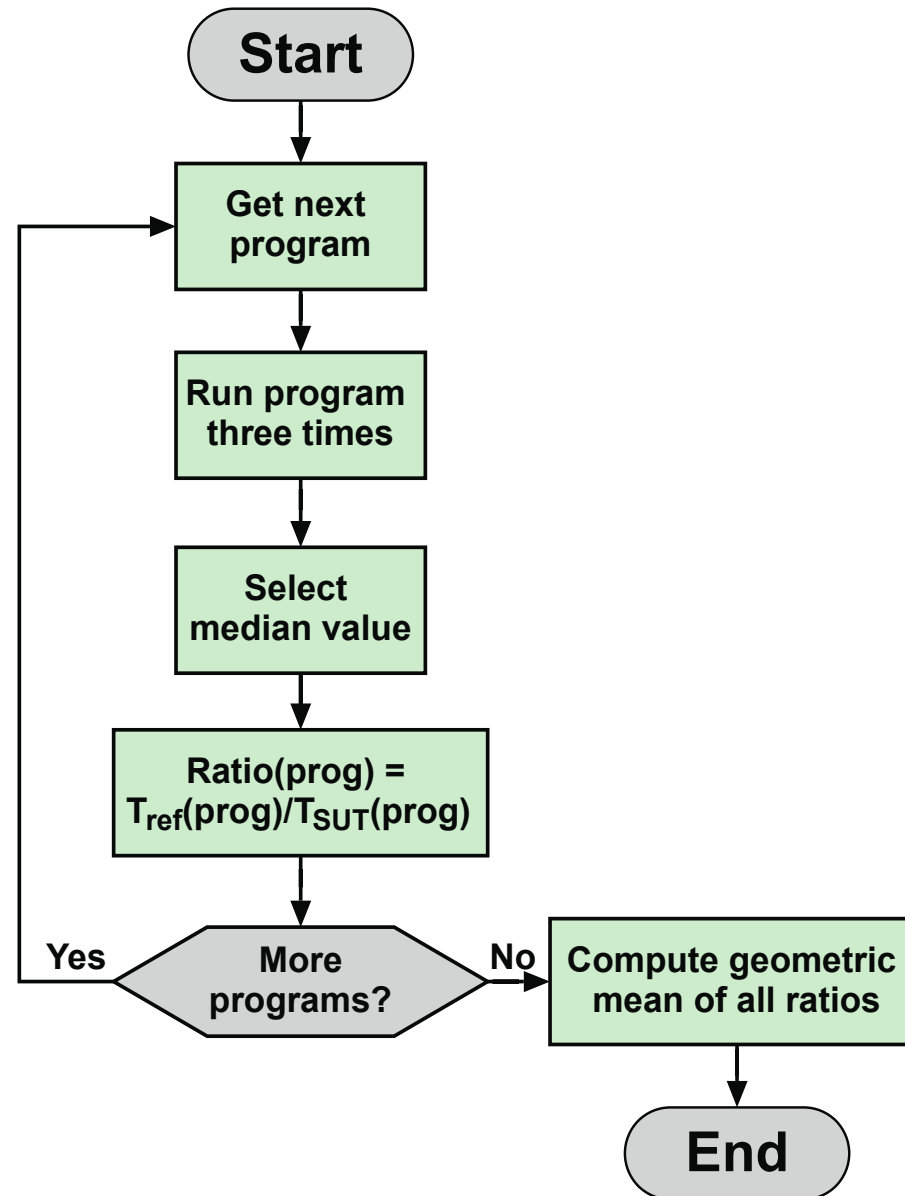
Benchmark	Reference time (hours)	Instr count (billion)	Language	Application Area	Brief Description
400.perlbench	2.71	2,378	C	Programming Language	PERL programming language interpreter, applied to a set of three programs.
401.bzip2	2.68	2,472	C	Compression	General-purpose data compression with most work done in memory, rather than doing I/O.
403.gcc	2.24	1,064	C	C Compiler	Based on gcc Version 3.2, generates code for Opteron.
429.mcf	2.53	327	C	Combinatorial Optimization	Vehicle scheduling algorithm.
445.gobmk	2.91	1,603	C	Artificial Intelligence	Plays the game of Go, a simply described but deeply complex game.
456.hmmer	2.59	3,363	C	Search Gene Sequence	Protein sequence analysis using profile hidden Markov models.
458.sjeng	3.36	2,383	C	Artificial Intelligence	A highly ranked chess program that also plays several chess variants.
462.libquantum	5.76	3,555	C	Physics / Quantum Computing	Simulates a quantum computer, running Shor's polynomial-time factorization algorithm.
464.h264ref	6.15	3,731	C	Video Compression	H.264/AVC (Advanced Video Coding) Video compression.
471.omnetpp	1.74	687	C++	Discrete Event Simulation	Uses the OMNet++ discrete event simulator to model a large Ethernet campus network.
473.astar	1.95	1,200	C++	Path-finding Algorithms	Pathfinding library for 2D maps.
483.xalanbmk	1.92	1,184	C++	XML Processing	A modified version of Xalan-C++, which transforms XML documents to other document types.

← (Left)
Table 2.5
(COAWS)
SPEC
CPU2006
Integer
Benchmarks

(Right) →
Table 2.6
(COAWS)
SPEC
CPU2006
Floating-point
Benchmarks

Benchmark	Reference time (hours)	Instr count (billion)	Language	Application Area	Brief Description
410.bwaves	3.78	1,176	Fortran	Fluid Dynamics	Computes 3D transonic transient laminar viscous flow.
416.gamess	5.44	5,189	Fortran	Quantum Chemistry	Quantum chemical computations.
433.milc	2.55	937	C	Physics / Quantum Chromodynamics	Simulates behavior of quarks and gluons
434.zeusmp	2.53	1,566	Fortran	Physics / CFD	Computational fluid dynamics simulation of astrophysical phenomena.
435.gromacs	1.98	1,958	C, Fortran	Biochemistry / Molecular Dynamics	Simulate Newtonian equations of motion for hundreds to millions of particles.
436.cactusADM	3.32	1,376	C, Fortran	Physics / General Relativity	Solves the Einstein evolution equations.
437.leslie3d	2.61	1,273	Fortran	Fluid Dynamics	Model fuel injection flows.
444.namd	2.23	2,483	C++	Biology / Molecular Dynamics	Simulates large biomolecular systems.
447.dealII	3.18	2,323	C++	Finite Element Analysis	Program library targeted at adaptive finite elements and error estimation.
450.soplex	2.32	703	C++	Linear Programming, Optimization	Test cases include railroad planning and military airlift models.
453.povray	1.48	940	C++	Image Ray-tracing	3D Image rendering.
454.calculix	2.29	3,041	C, Fortran	Structural Mechanics	Finite element code for linear and nonlinear 3D structural applications.
459.GemsFDTD	2.95	1,320	Fortran	Computational Electromagnetics	Solves the Maxwell equations in 3D.
465.tonto	2.73	2,392	Fortran	Quantum Chemistry	Quantum chemistry package, adapted for crystallographic tasks.
470.lbm	3.82	1,500	C	Fluid Dynamics	Simulates incompressible fluids in 3D.
481.wrf	3.10	1,684	C, Fortran	Weather	Weather forecasting model
482.sphinx3	5.41	2,472	C	Speech recognition	Speech recognition software.

SPEC BENCHMARKING WORKFLOW



TERMS USED IN SPEC DOCUMENTATION

- **Benchmark**
 - A program written in a high-level language that can be compiled and executed on any computer that implements the compiler.
- **System under test**
 - This is the system to be evaluated.
- **Reference machine**
 - This is a system used by SPEC to establish a baseline performance for all benchmarks
 - Each benchmark is run and measured on this machine to establish a reference time for that benchmark
- **Base metric**
 - These are required for all reported results and have strict guidelines for compilation
- **Peak metric**
 - This enables users to attempt to optimize system performance by optimizing the compiler output.
- **Speed metric**
 - This is simply a measurement of the time it takes to execute a compiled benchmark
 - Used for comparing the ability of a computer to complete single tasks
- **Rate metric**
 - This is a measurement of how many tasks a computer can accomplish in a certain amount of time
 - This is called a throughput, capacity, or rate measure
 - Allows the system under test to execute simultaneous tasks to take advantage of multiple processors

EXAMPLE PROBLEM

- IS SPEC Rating Misleading too?

Metrics	Computer A	Computer B	Computer C	CPU Time ↓	SPEC Rating ↑
Program X	1	0.2	2		
Program Y	2	4	0.4		
<i>Total Ex Time</i>	<i>3</i>	<i>4.2</i>	<i>2.4</i>	<i>Computer C</i>	
<i>SPEC (Tref=A)</i>	<i>1</i>	<i>1.581138</i>	<i>1.581138</i>	<i>Computer B.C</i>	<i>GM</i>
	<i>1</i>	<i>2.75</i>	<i>2.75</i>	<i>Computer B</i>	<i>AM</i>

[Reference to the paper: LINK](#)

EXAMPLE PROBLEM

- IS SPEC Rating Misleading too?

Program X = 1 billion instructions
Program Y = 4 billion instructions

Machine A
CPI (Program X) = 1
CPI (Program Y) = 2
Clockrate = 1Ghz

Machine B
CPI (Program X) = 8
CPI (Program Y) = 1
Clockrate = 2Ghz

Machine C
CPI (Program X) = 0.25
CPI (Program Y) = 8
Clockrate = 1Ghz

Metrics	Computer A	Computer B	Computer C	CPU Time ↓	SPEC Rating ↑
Program X	1	4	0.25		
Program Y	8	2	32		
<i>Total Ex Time</i>	<i>9</i>	<i>6</i>	<i>32.25</i>	<i>Computer B</i>	
<i>SPEC (Tref=A) GM</i>	<i>1</i>	<i>1 (0.25, 4)</i>	<i>1 (4, 0.25)</i>	<i>Computer A.B.C</i>	<i>GM</i>
<i>SPEC (Tref=A) AM</i>	<i>1</i>	<i>2.125</i>	<i>2.125</i>	<i>Computer B.C</i>	<i>AM</i>
<i>SPEC (Tref=A) HM</i>	<i>1</i>	<i>0.470588</i>	<i>0.470588</i>	<i>Computer A</i>	<i>HM</i>

[Reference to the paper: LINK](#)

EXAMPLE PROBLEM

- IS SPEC Rating Misleading too?

Metrics	Computer A	Computer B	Computer C	CPU Time 	
Program W	1	0.5	1.5		
Program X	2	2.5	2.5		
Program Y	3	3.5	2.5		
Program Z	4	3	2.75		
Total Ex Time	10	9.5	9.25	Computer C	
SPEC (Tref=A)	1	1.16286	0.98226	Computer B	GM
	1	1.2476	1.0303	Computer B	AM

$$\text{CPU Time} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Clock cycles}}{\text{Instruction}} \times \frac{\text{Seconds}}{\text{Clock cycle}}$$

- Energy efficiency of a benchmark program is given by:

$$\text{Energy efficiency} = \frac{1/(\text{Execution time})}{\text{Power in watts}}$$

$$= \text{Program units/joule}$$

- Relative efficiency:

$$\text{Relative efficiency} = \frac{\text{Efficiency of a computer}}{\text{Eff. of reference computer}}$$

- MIPS & MFLOPS: (Measure of # of Million Instructions executed per second)

EXAMPLE PROBLEM

- Why MIPS Rating can be misleading indicator for performance?

Measurement	Computer A	Computer B	CPU Time ↓	MIPS ↑
Instruction Count	10 billion	8 billion		
Clock Rate	4 Ghz	4 GHz		
CPI	1.0	1.1		
<i>Winner</i>			<i>Computer ??</i>	<i>Computer ??</i>

$$\text{CPU Time} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Clock cycles}}{\text{Instruction}} \times \frac{\text{Seconds}}{\text{Clock cycle}}$$

EXAMPLE PROBLEM

- Why MIPS Rating can be misleading indicator for performance?

Measurement	Computer A	Computer B	CPU Time ↓	MIPS ↑
Instruction Count	10 billion	8 billion	A = 2.5s	A = 4000
Clock Rate	4 Ghz	4 GHz	B = 2.2s	B = 3636
CPI	1.0	1.1		
<i>Winner</i>			<i>Computer ??</i> B	<i>Computer ??</i> A

$$\text{CPU Time} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Clock cycles}}{\text{Instruction}} \times \frac{\text{Seconds}}{\text{Clock cycle}}$$

$$\text{MIPS} = IC / (\text{Execution Time} \times 10^6)$$

ANOTHER EXAMPLE (COAWS)

- Why MIPS Rating can be misleading indicator for performance?

A = B+C ; / Assume (A,B,C) are all in memory*/*

Intel Processor:

add mem(b) mem(c) mem(a)

Instruction Count = 1

Cycles Per Instruction = 8

ARM Processor:

load mem(b), reg(1)

load mem(c), reg(2)

add reg(1), reg(2), reg(3)

Store reg(3), mem(a)

Instruction Count = 4

Cycles Per Instruction = 2

$$\text{CPU Time} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Clock cycles}}{\text{Instruction}} \times \frac{\text{Seconds}}{\text{Clock cycle}}$$

$$\text{MIPS} = \text{IC} / (\text{Execution Time} \times 10^6)$$

Measurement	Intel Processor	ARM Processor	CPU Time	MIPS
Instruction Count	1	4		
Clock Rate	4 Ghz	4 GHz		
CPI	8	2		
Winner			Computer ??	Computer ??

Suppose we enhance a machine by making all floating-point instructions run 5 times faster. Now suppose the marketing team wants to showcase the new floating point unit enhancement and desires to show at least a speed up of 3. Then, what percentage of the execution time would floating-point instructions must account in order to yield our desired speedup on this benchmark?

Suppose we enhance a machine by making all floating-point instructions run **5** times faster. If the execution time of some benchmark program before the floating-point enhancement is **10** seconds. What will the speedup be if only $\frac{1}{2}$ of the **10** seconds is spent executing floating-point instructions?

PERFORMANCE SUMMARY

- To Summarize:
 - What is Computer Architecture and Organization all about?
 - Birds eye view of study on COA – Black Box; Grey Box and White Box
 - Key Ideas behind Computer Architecture and the Technical advancements;
 - What do we mean by Performance and associated performance metrics.
 - How to evaluate and express the Performance of a program and computer system.
- Some related topics to be addressed:
 - Technological Advancements and their implications on computing systems
 - Power Wall.
 - Amdahl's Law – *Most relevant and we can discuss in next lecture.*
 - Chip Manufacturing/Yield – *Typically covered as first chapter in VLSI course.*

Reading Materials: Chapter 1 (COAHP) and Chapters 1, 2 (COAWS)

Checkout this Video:

Integrated Circuits Design and Manufacturing

EVOLUTION OF COMPUTERS – THE 3Ps

Year	Name	Size (cu. ft.)	Power (watts)	Performance (adds/sec)	Memory (KB)	Price	Price/ performance vs. UNIVAC	Adjusted price (2007 \$)	Adjusted price/ performance vs. UNIVAC
1951	UNIVAC I	1,000	125,000	2,000	48	\$1,000,000	1	\$7,670,724	1
1964	IBM S/360 model 50	60	10,000	500,000	64	\$1,000,000	263	\$6,018,798	319
1965	PDP-8	8	500	330,000	4	\$16,000	10,855	\$94,685	13,367
1976	Cray-1	58	60,000	166,000,000	32,000	\$4,000,000	21,842	\$13,509,798	47,127
1981	IBM PC	1	150	240,000	256	\$3,000	42,105	\$6,859	134,208
1991	HP 9000/ model 750	2	500	50,000,000	16,384	\$7,400	3,556,188	\$11,807	16,241,889
1996	Intel PPro PC (200 MHz)	2	500	400,000,000	16,384	\$4,400	47,846,890	\$6,211	247,021,234
2003	Intel Pentium 4 PC (3.0 GHz)	2	500	6,000,000,000	262,144	\$1,600	1,875,000,000	\$2,009	11,451,750,000
2007	AMD Barcelona PC (2.5 GHz)	2	250	20,000,000,000	2,097,152	\$800	12,500,000,000	\$800	95,884,051,042

FIGURE e1.12.7 Characteristics of key commercial computers since 1950, in actual dollars and in 2007 dollars adjusted for inflation. The last row assumes we can fully utilize the potential performance of the four cores in Barcelona. In contrast to [Figure e1.12.3](#), here the price of the IBM S/360 model 50 includes I/O devices. (Source: *The Computer History Museum and Producer Price Index for Industrial Commodities.*)

GENERATIONS OF COMPUTERS

Table 1.2 Computer Generations

Generation	Approximate Dates	Technology	Typical Speed (operations per second)
1	1946–1957	Vacuum tube	40,000
2	1957–1964	Transistor	200,000
3	1965–1971	Small- and medium-scale integration	1,000,000
4	1972–1977	Large scale integration	10,000,000
5	1978–1991	Very large scale integration	100,000,000
6	1991–	Ultra large scale integration	>1,000,000,000

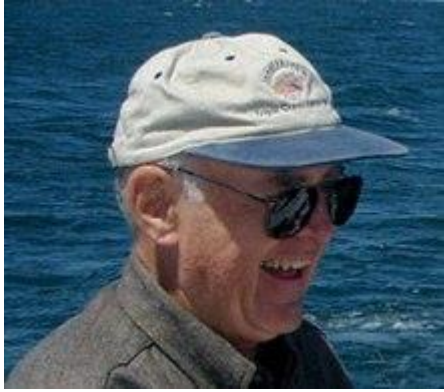
<https://www.cpu-world.com/index.html>

<https://arstechnica.com/business/news/2010/02/two-billion-transistor-beasts-power7-and-niagara-3.ars>

<https://hackaday.com/2019/08/21/largest-chip-ever-holds-1-2-trillion-transistors/>

GORDON MOORE VS ARTHUR ROCK

Gordon Moore

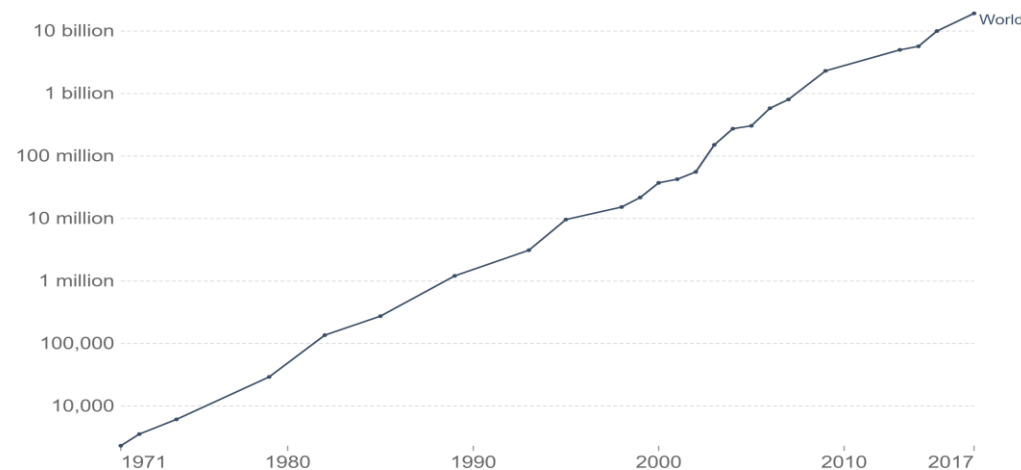


The number of transistors on microchips in a dense IC doubles almost about every two years.

Moore's Law: Transistors per microprocessor

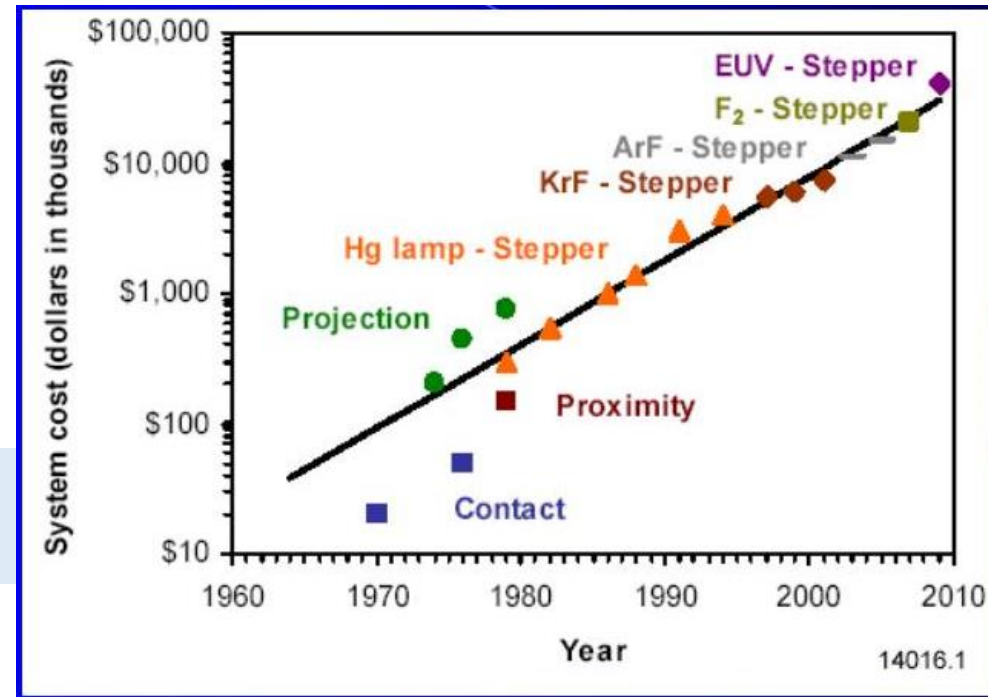
Number of transistors which fit into a microprocessor. This relationship was famously related to Moore's Law, which was the observation that the number of transistors in a dense integrated circuit doubles approximately every two years.

Our World in Data

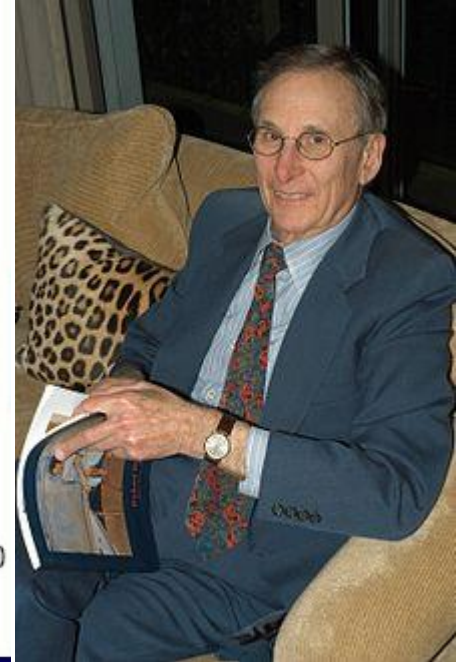


Source: Karl Rupp. 40 Years of Microprocessor Trend Data.

CC BY



Arthur Rock



The cost of a [semiconductor chip fabrication plant](#) doubles almost every four years.

- In 1986, x386 with 0.25 million transistors cost \$200 million
- In 1996, Pentium with 6 million transistors cost \$2000 M
- In (~2016) stats with billion transistors cost ~\$15 Billion

Our World
in Data

Transistor count



GENE AMDAHL VS JOHN GUSTAFFSON

Gene Amdahl



$$speedup(P, N) = \frac{1}{(1 - P) + \frac{P}{N}}$$

$$speedup(P, N) = \frac{N}{N + P * (1 - N)}$$

$$speedup(P, N) = \frac{N}{1 + (1 - P) * (N - 1)}$$

P = parallelizable portion

s = (1-P) serial portion

N = Parallel units of execution

As N tends to ∞ speedup = $1/(1-P)$

$$speedup(P, N) = (1 - P) + P * N$$

$$= N + (1 - N) * (1 - P)$$

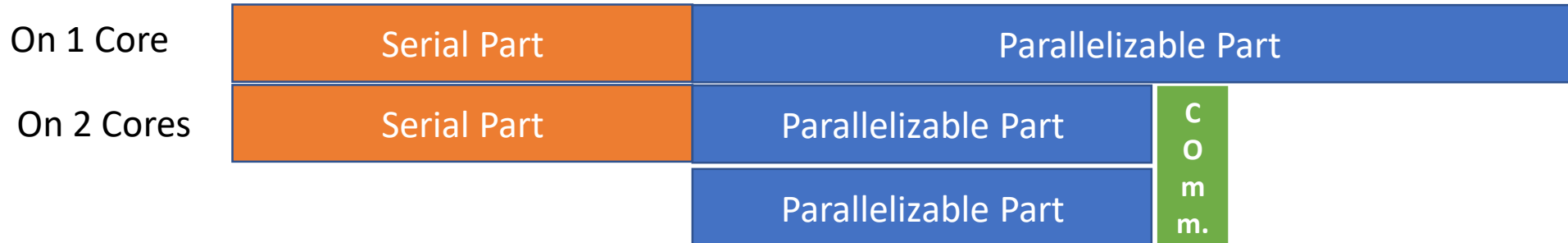
John L Gustafson



$$Efficiency(N) = Speedup(P, N)/N$$

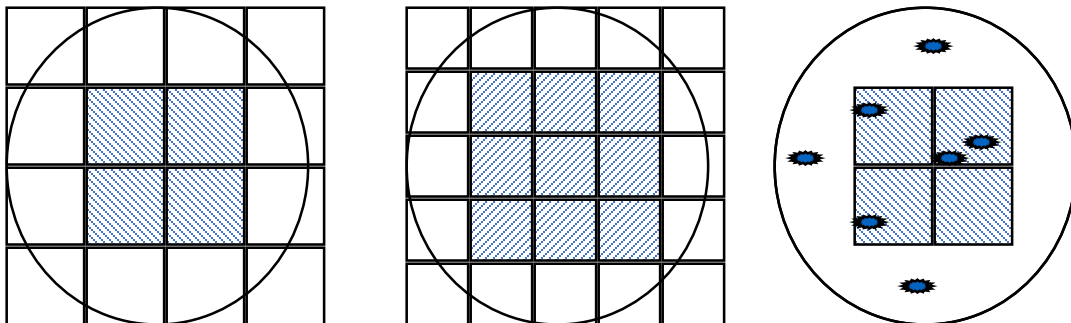
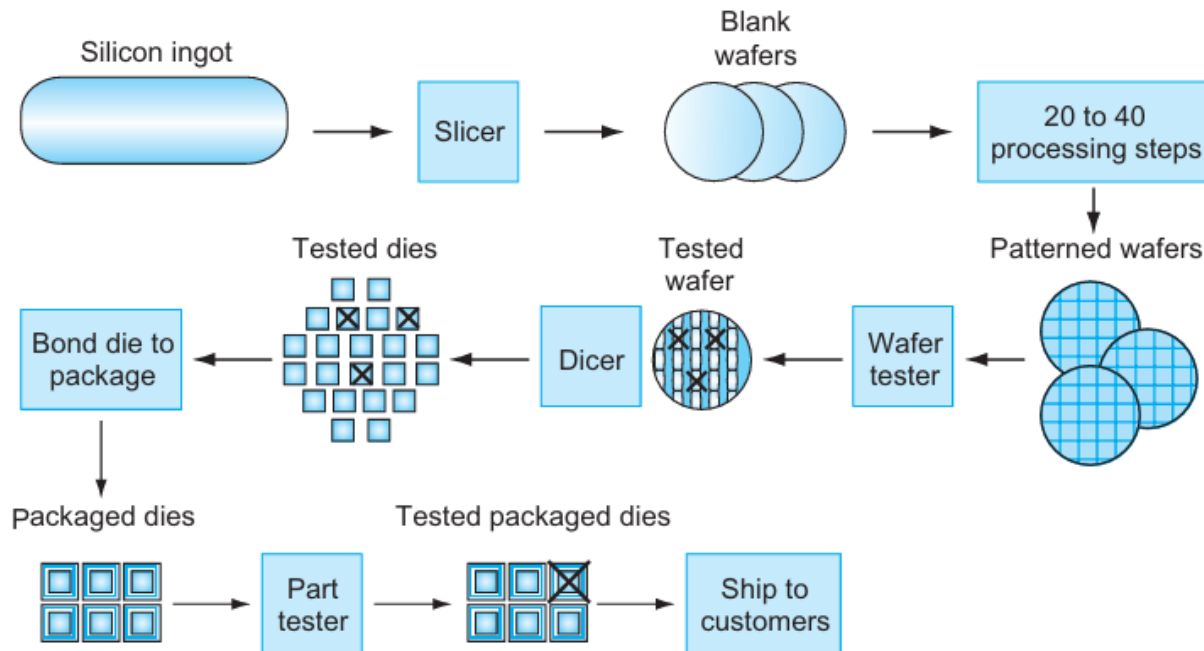
Pessimistic law: (Fixed size of the program) -- theoretical **Maximum speedup is rather limited by the sequential part of the program**

Optimistic law: (Fixed time of the Program) – theoretical maximum speedup is the same, but the amount of work done can be significantly increased.



COST OF INTEGRATED CIRCUITS

• Chip Manufacturing:



$$\text{IC cost} = \frac{\text{Die cost} + \text{Testing cost} + \text{Packaging cost}}{\text{Final test yield}}$$

$$\text{Die cost} = \frac{\text{Wafer cost}}{\text{Dies per Wafer} \times \text{Die yield}}$$

$$\text{Dies per wafer} = \frac{\text{Wafer_Area}}{\text{Die_Area}}$$

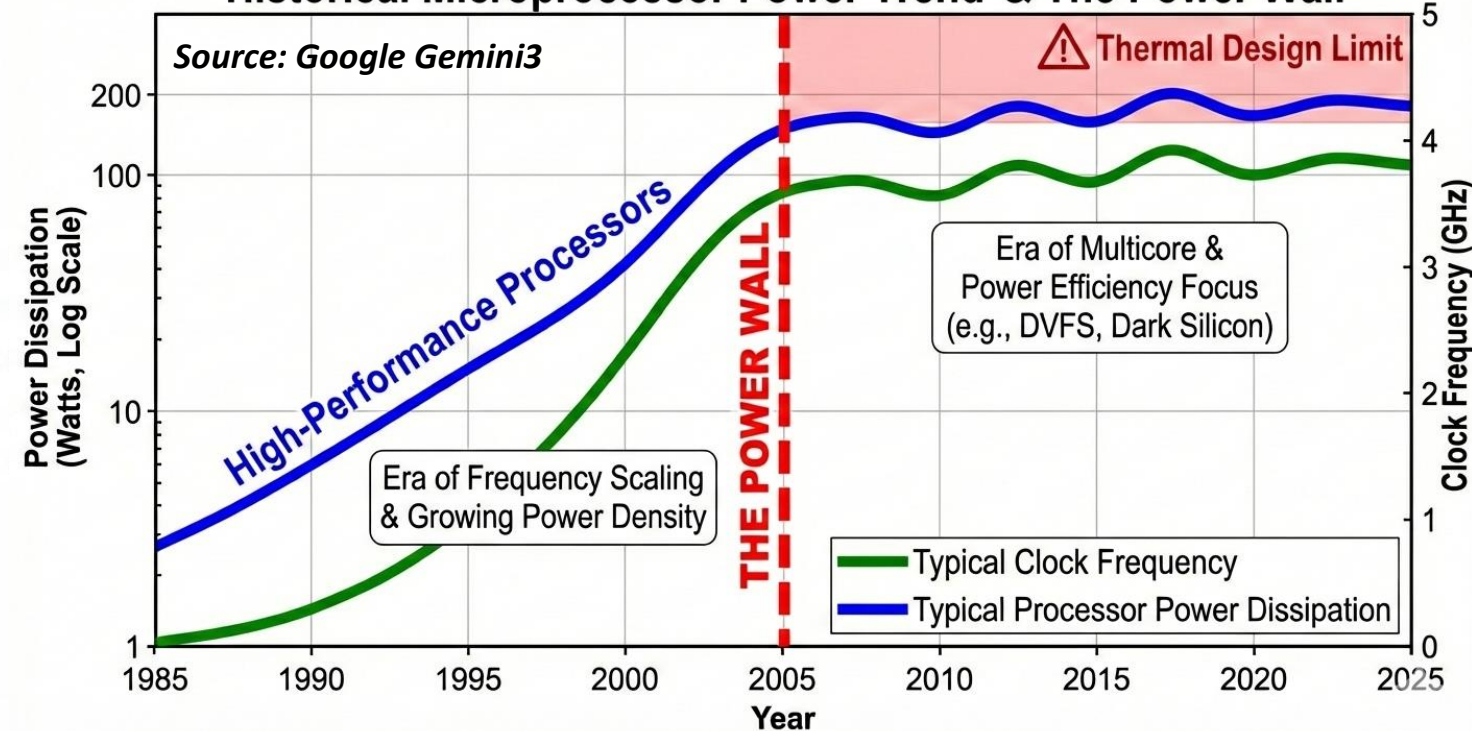
$$\text{Die Yield} = 1 / \left\{ 1 + \left(\frac{\text{Defect_Density} \times \text{Die_Area}}{\alpha} \right) \right\}^{-\alpha}$$

- Typical value of α is 2.

POWER

- Power: A critical constraint in μ processor design.

Historical Microprocessor Power Trend & The Power Wall



- Power consumption represents the rate at which electrical energy is converted into heat by the processor.
- Impacts: Battery life, Operating Costs, Heat dissipation, Reliability, Efficiency and Environment.
- Hence, managing power consumption of processors is critical.

- Two components: **Static** and **Dynamic**

$$P_{total} = P_{static} + P_{dynamic}$$

- Static Power** (Leakage power) is consumed when the processor is idle (no activity).

$$P_{static} = V_{dd} \cdot I_{leakage}$$

- Dynamic Power:** when transistors inside the processor actively switch (1 to 0, vice-versa)

$$P_{dynamic} = P_{switching} + P_{shortcircuit}$$

$$P_{switching} = \alpha \cdot C_L V_{dd}^2 \cdot f$$

α = Activity factor (usually ≤ 0.5)

$$P_{shortcircuit} = \frac{\beta}{12} (V_{dd} - 2V_{th})^3 \cdot \tau f$$

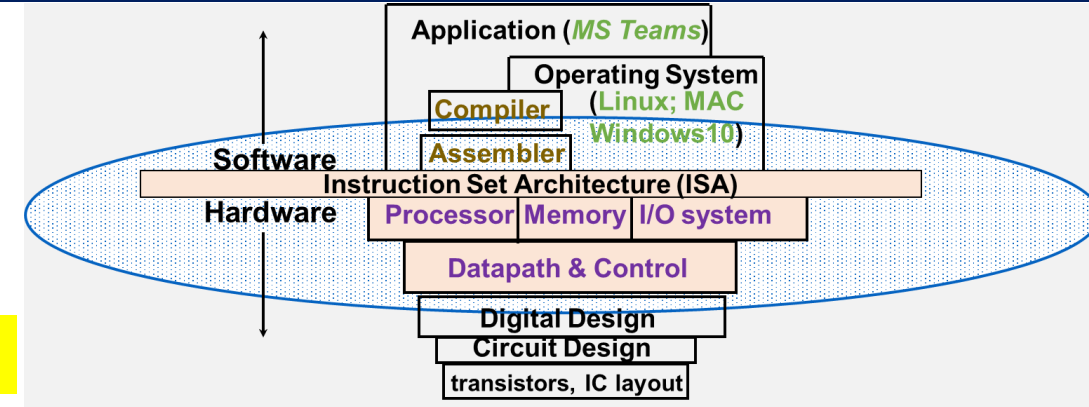
β = Transistor Gain Factor
 τ = input signal rise/fall time

Power Wall: Thermal limit on frequency scaling. *i.e.* increasing performance (by scaling up frequency or transistors) becomes impractical due to power consumption and heat dissipation.

COMPUTER ARCHITECTURE –INSTRUCTION SET ARCHITECTURE

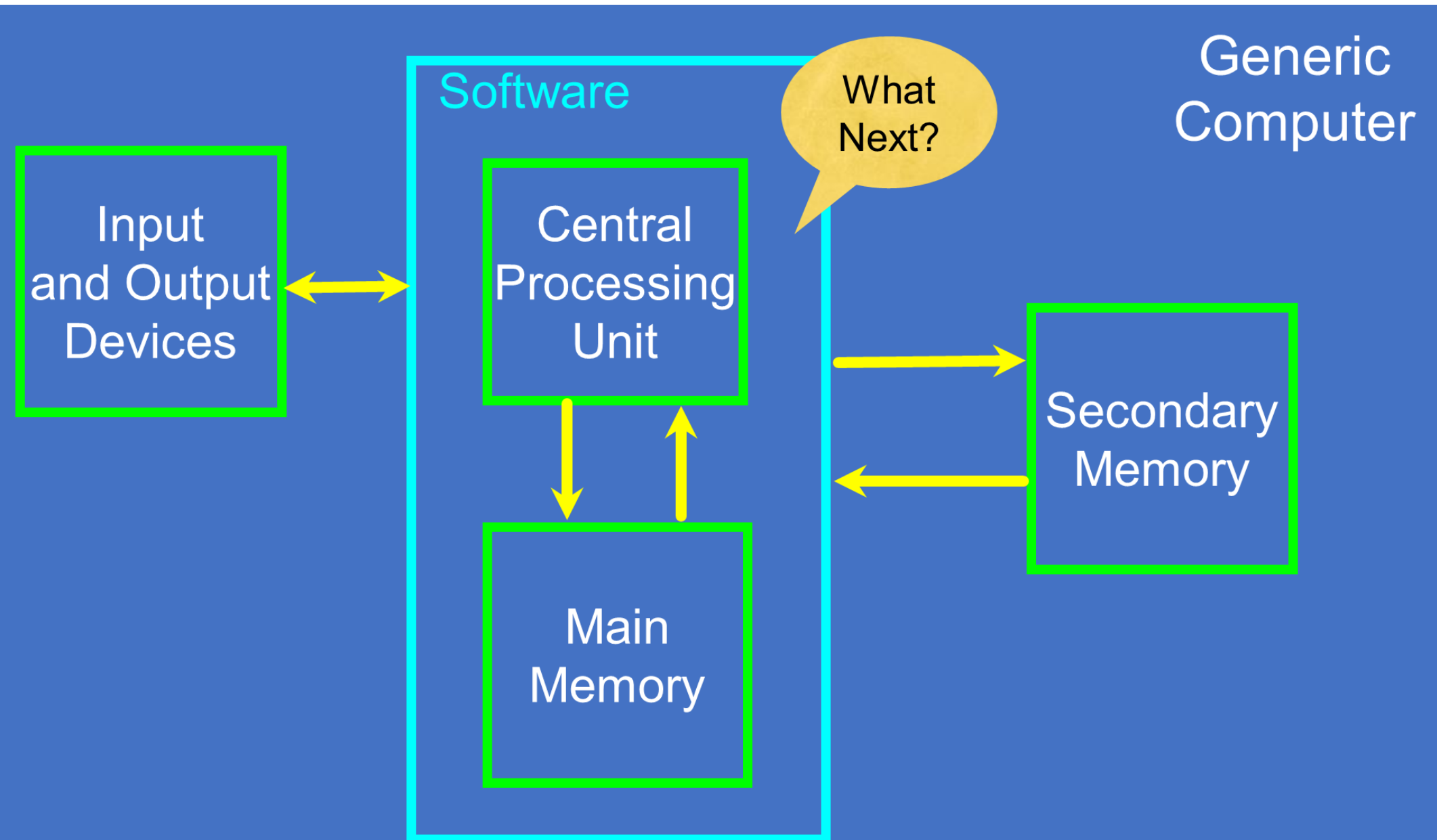
20-Jan-2026

- Recap:
 - What is COA? Architecture vs Microarchitecture
 - Performance: Latency, Throughput and Speedup
 - Historical perspectives – Evolution and Basic Laws
- This/Next weeks Agenda (~next 4-5 lectures):
 - High level Computer Organization
 - Dive into Computer Architecture.
 - Instruction set Architecture (ISA)
 - (MIPS – case study) & Glance over other ISAs
- Logistics:
 - Quiz-01 and Assignmnet-01 (posted)
 - Additional reading/reference will be posted.



- Reading Materials:
 - Chapter 2 - COAHP
 - Chapters 3, 12 - COAWS

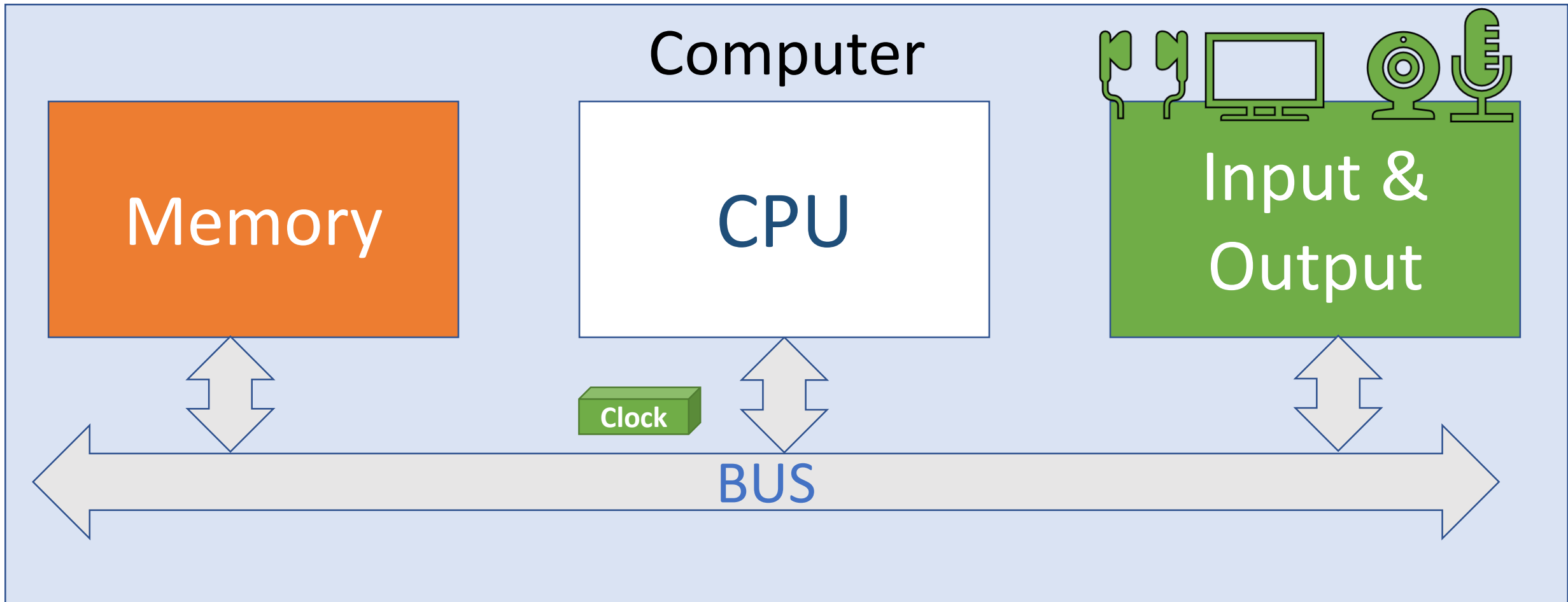
HOW DOES A PROGRAM EXECUTE?



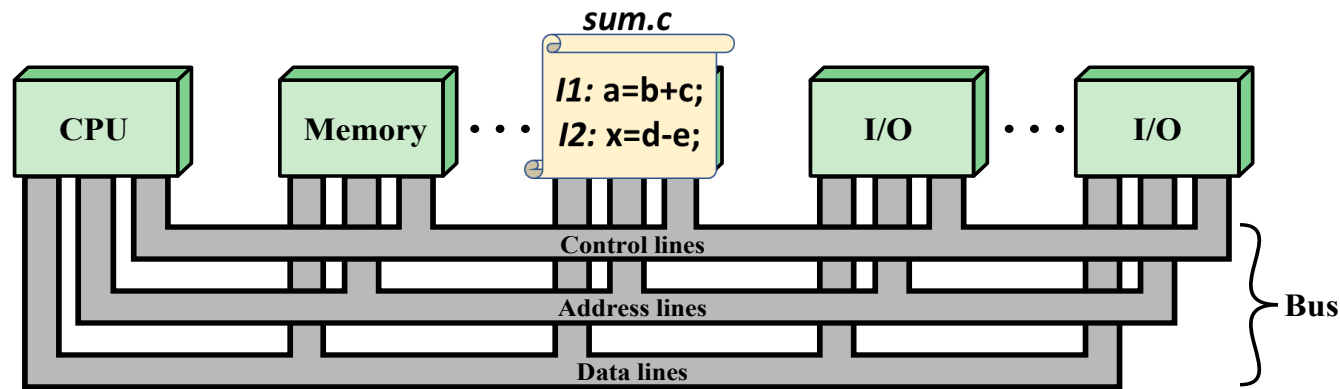
sum.c

```
I1: a=b+c;  
I2: x=d-e;
```


HIGH-LEVEL COMPONENTS OF A COMPUTER



PEELING ONE LAYER OUT.. ONE LAYER INTO THE COMPUTER



Memory address register (MAR)

- Specifies the address in memory for the next read or write

Memory buffer register (MBR)

- Contains the data to be written into memory or receives the data read from memory

I/O address register (I/OAR)

- Specifies a particular I/O device

I/O buffer register (I/OBR)

- Used for the exchange of data between an I/O module and the CPU

Memory to processor

Processor reads an instruction or a unit of data from memory

Processor to memory

Processor writes a unit of data to memory

I/O to processor

Processor reads data from an I/O device via an I/O module

Processor to I/O

Processor sends data to the I/O device

I/O to or from memory

An I/O module is allowed to exchange data directly with memory without going through the processor using direct memory access